The BPA Lab

Architecture and Documentation of a Demonstration Factory for Business Process Automation and Analytics

Matthias Zapp

May 26, 2026

Table of contents

1. Overview

This book documents the **Business Process Automation Lab (BPA Lab)** at TH Köln — a small-scale, modular model factory that demonstrates and teaches concepts and technologies for business process automation and analytics.

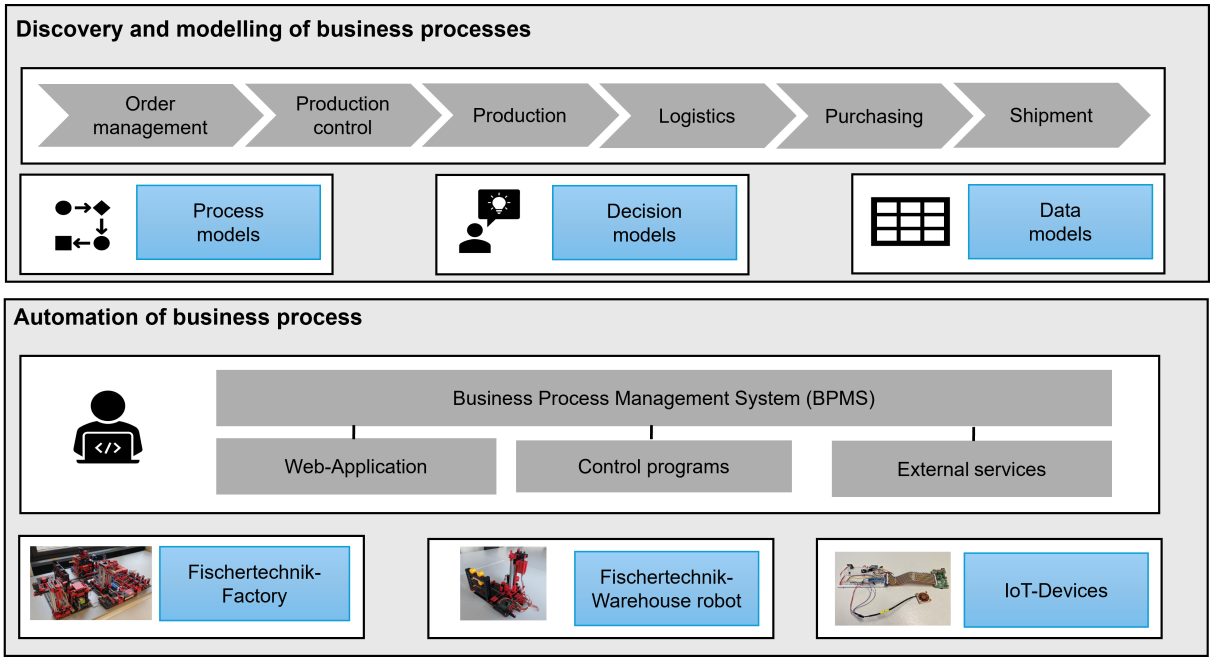

Figure 1.1.: BPA Lab — Conceptual Architecture

1.1. About this book

The BPA Lab combines **software components** (Business Process Management Systems, Robotic Process Automation, Process Mining) with **hardware components** (a Fischertechnik Industry 4.0 factory, a Fischertechnik warehouse robot, and IoT devices) to simulate the production and logistics processes of a bicycle manufacturer.

This book aims to provide:

1. A **high-level overview** of the BPA Lab and its goals.
2. A **description of the business scenario** — the end-to-end ordering, manufacturing, and shipment process implemented in the lab.
3. A **description of the software architecture** and the design decisions taken along the way.

It is structured as follows:

1. **Introduction:** Motivation, goals, and scope of the BPA Lab.
2. **Solution overview:** The three architectural layers — controllers, process applications, workflow engine.
3. **Business scenario:** The end-to-end bicycle ordering, manufacturing, and shipment process, broken down into its sub-processes (order management, production control, purchasing, manufacturing, shipment, warehouse operations).
4. **Data and data sources:** Data model, data flows, and persistence in the BPA Lab.
5. **Software architecture:** Requirements and a C4 view of the system.
6. **Architecture decisions:** Documented decisions and their rationales.

The appendices contain:

- **Appendix A:** Overview of related components and sister repositories.
- **Appendix B:** A glossary of key terms.
- **References:** All cited literature.

1.2. Who this book is for

This book is written for:

- **Lecturers and researchers** who want to teach business process automation with a tangible, end-to-end reference system.
- **Students** working on bachelor's or master's projects inside the BPA Lab.
- **Practitioners** looking for a concrete reference architecture combining a BPMS, IoT hardware, and process mining.

1.3. Source code and related repositories

The implementation of the demonstration factory is maintained as a separate repository: bpa_lab_demonstration_factory.

Documentation for student component projects is available in the wiki of bpa_lab_student_docs.

1.4. Status and roadmap

> **!** Work in progress
>
> The BPA Lab — and this book — are under continuous development. Several sections are marked as *work in progress* and will be expanded in future revisions. Bug reports and suggestions are very welcome via the issue tracker.

1.5. Licence and citation

Suggested citation:

> Zapp, Matthias (2026). *The BPA Lab — Architecture and Documentation of a Demonstration Factory for Business Process Automation and Analytics.* TH Köln (University of Applied Sciences). Online: https://bpalabthcologne.github.io/bpa_lab_book/.

2. Introduction

> **Learning objectives**
>
> After reading this chapter, you will be able to:
>
> - describe the purpose and main goals of the BPA Lab,
> - distinguish between its *demonstration* and *teaching* missions,
> - identify the main hardware and software components and their roles.

2.1. What is the BPA Lab?

The **Business Process Automation Lab (BPA Lab)** at TH Köln is a small, modular model factory dedicated to business process automation and analytics. It combines real-world software components used in industry — Business Process Management Systems (BPMS), Robotic Process Automation (RPA), and process-mining tools — with physical hardware: a Fischertechnik Industry 4.0 factory, a Fischertechnik warehouse robot, and a range of IoT devices.

Together, these components simulate the **production and logistics processes of a bicycle manufacturer** — from the customer order to the delivery of a custom-built bike.

2.2. Two goals: demonstration and teaching

The BPA Lab pursues two complementary goals.

2.2.1. Demonstration

The lab demonstrates concepts and technologies for business process automation and analytics in a tangible way. It shows how:

- a central BPMS (Camunda 8) orchestrates end-to-end business processes,
- modular **job workers** implement the actual work and interact with hardware,
- **process mining** can be used to analyse process executions and identify improvement opportunities.

By bringing together software and hardware in one running system, the lab makes abstract concepts immediately visible.

2.2.2. Teaching

Individual building blocks of the BPA Lab — for instance the Fischertechnik warehouse robot — are also used in **student projects** (bachelor's and master's level) as starting points for self-defined process implementations. This enables a hands-on form of teaching for **process-oriented application integration**: students do not merely model processes on paper but see them execute in a real, physical system.

2.3. Why a demonstration factory?

Business process management theory and notations like BPMN are often taught using abstract examples — pizza ordering, expense approvals, generic procurement workflows. These examples are accessible but rarely convey:

- the **technical complexity** of integrating heterogeneous systems,
- the **operational concerns** of running long-lived process instances,
- the **data perspective** that is at the heart of process mining and analytics.

A demonstration factory bridges this gap. Students and visitors can observe how a customer order arrives via a web form, triggers a workflow in Camunda 8, dispatches commands via MQTT to physical hardware, and eventually leads to a real, manufactured product being moved through a real, physical warehouse.

2.4. A bird's-eye view

The next chapter — Solution overview — introduces the three architectural layers of the BPA Lab. Subsequent chapters describe the business scenario, the data perspective, the software architecture, and the architecture decisions taken so far.

> **How to read this book**
>
> If you are new to the BPA Lab, read the chapters in order. If you are already familiar with the lab and want to consult a specific aspect — for example a particular sub-process or an architectural decision — feel free to jump directly to the relevant chapter using the sidebar or the search box at the top of the page.

3. Solution Overview

Learning objectives

After reading this chapter, you will be able to:

- name the three architectural layers of the BPA Lab and explain their responsibilities,
- describe the role of Camunda 8 as the central workflow engine,
- explain how hardware components and process applications interact,
- justify the modular, domain-driven design approach used in the BPA Lab.

3.1. Three architectural layers

The architecture and implementation of the BPA Lab are under continuous development. The solution is structured into three clearly separated layers.

3.1.1. Layer 1 — Controllers

Controllers are responsible for the direct control of hardware components: the Fischertechnik Industry 4.0 factory, the Fischertechnik warehouse robots, and IoT devices. They are predominantly implemented in **Python** and translate high-level commands (for example *"transport workpiece from slot A to slot B"*) into the low-level signals that move motors and read sensors.

3.1.2. Layer 2 — Process applications (job workers)

Process applications are middleware components that implement the actual business logic of individual work steps. In Camunda 8 terms, they are **job workers**: they subscribe to specific service tasks and execute them when activated by the workflow engine.

Job workers communicate:

- with the **workflow engine** via **gRPC** (the standard Camunda 8 / Zeebe protocol),
- with the **controllers** via **MQTT** (a lightweight publish-subscribe protocol that is well established in the IoT space).

Process applications are predominantly implemented in **Java**.

3.1.3. Layer 3 — BPMS workflow engine

The central workflow engine is **Camunda 8 (self-managed)**. It executes:

- **BPMN** process models — describing how work flows through the system,
- **DMN** decision models — encoding business rules.

Camunda 8 maintains the state of each running process instance, handles incidents, and provides the data foundation for process mining and analytics.

3.2. A modular, domain-driven design

A key design principle in the BPA Lab is **modularity**. The system follows a **Domain-Driven Design (DDD)** approach so that:

- **multiple students and developers can work in parallel** on different parts of the system without stepping on each other's toes,
- **components can be reused** in student projects to build new process applications,
- **components can be replaced or upgraded** independently.

The primary criterion for splitting job workers and process models is the **business domain**:

- Order management
- Manufacturing
- Shipping
- Purchasing
- Logistics / warehouse operations

This separation aligns with the bounded contexts familiar from DDD and makes the system easier to reason about, both for teachers explaining it and for students extending it.

3.3. A simplified bird's-eye view

The following diagram summarises the interaction between the three layers and the hardware they ultimately drive.

The customer-facing entry point is typically a **web application** for order capture. From there, work flows downwards through Camunda 8 and the job workers and eventually reaches the physical Fischertechnik components. Events generated along the way flow back upwards and are persisted in databases for later analysis.

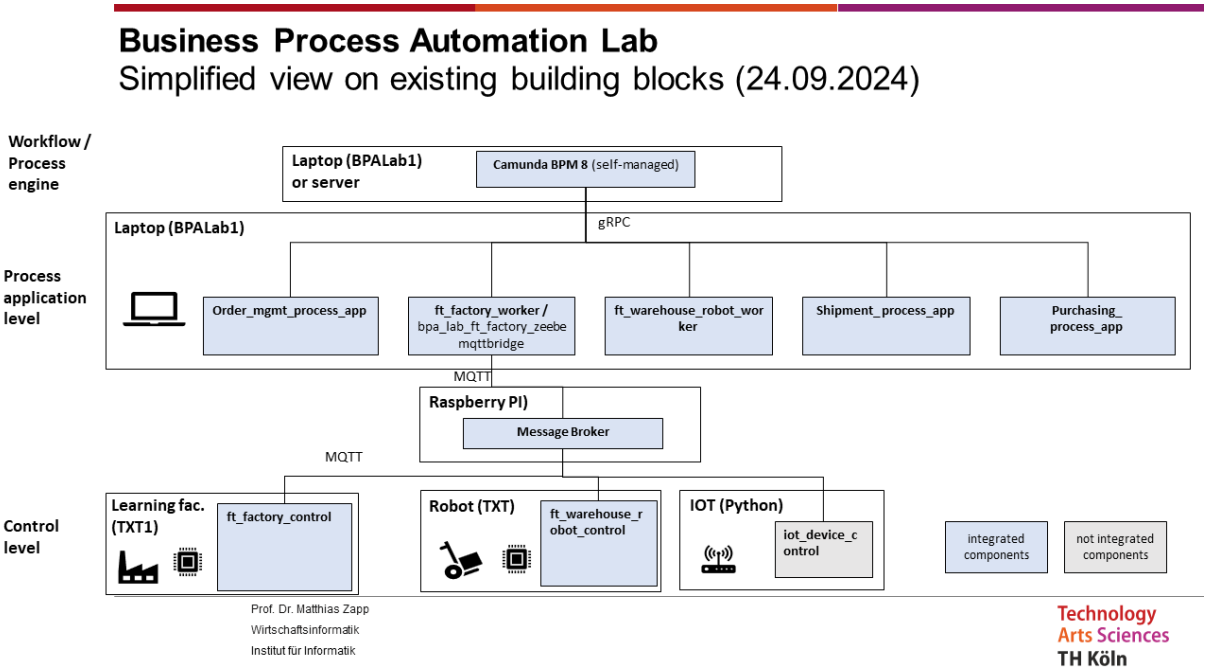

Figure 3.1.: Simplified architecture of the BPA Lab

3.4. The demonstration factory implementation

The latest version of the BPA Lab as a runnable **demonstration factory** is available in a separate repository:

github.com/BpaLabTHCologne/bpa_lab_demonstration_factory

The implementation is fully containerised with **Docker**. This makes local setup straightforward and ensures that the environment is reproducible across different machines — a key requirement when students set up the system on their own laptops.

4. Business Scenario of the Demonstration Factory

Learning objectives

After reading this chapter, you will be able to:

- describe the end-to-end business processes of a custom bicycle manufacturer in BPMN terms,
- explain the orchestrating role of the order management process,
- distinguish between the responsibilities of the order management, production control, purchasing, manufacturing, shipment, and warehouse operations processes,
- recognise how Fischertechnik hardware is used to *physically* execute parts of these processes.

Status of this chapter

This chapter describes the business processes of the BPA Lab at a strategic level. It is *work in progress* — terminology, level of detail, and harmonisation of process models are being refined as the lab evolves.

4.1. The scenario in a nutshell

A bicycle manufacturer offers highly customisable bicycles to consumers.

The entire end-to-end process is orchestrated by the **order management process**, which in turn initiates purchasing, manufacturing, and shipping processes as needed.

The manufacturing processes use the **Fischertechnik Industry 4.0 factory** to post goods receipts of components and to perform the actual manufacturing steps. Order management and shipping additionally use the **distribution warehouse** (a separate Fischertechnik robot) to post goods receipts and goods issues for finished products.

The following sections describe each sub-process in turn. The process models do **not** include technical aspects and show only the **happy path** of execution.

4.2. Order management

The order management process governs the fulfilment of customer orders.

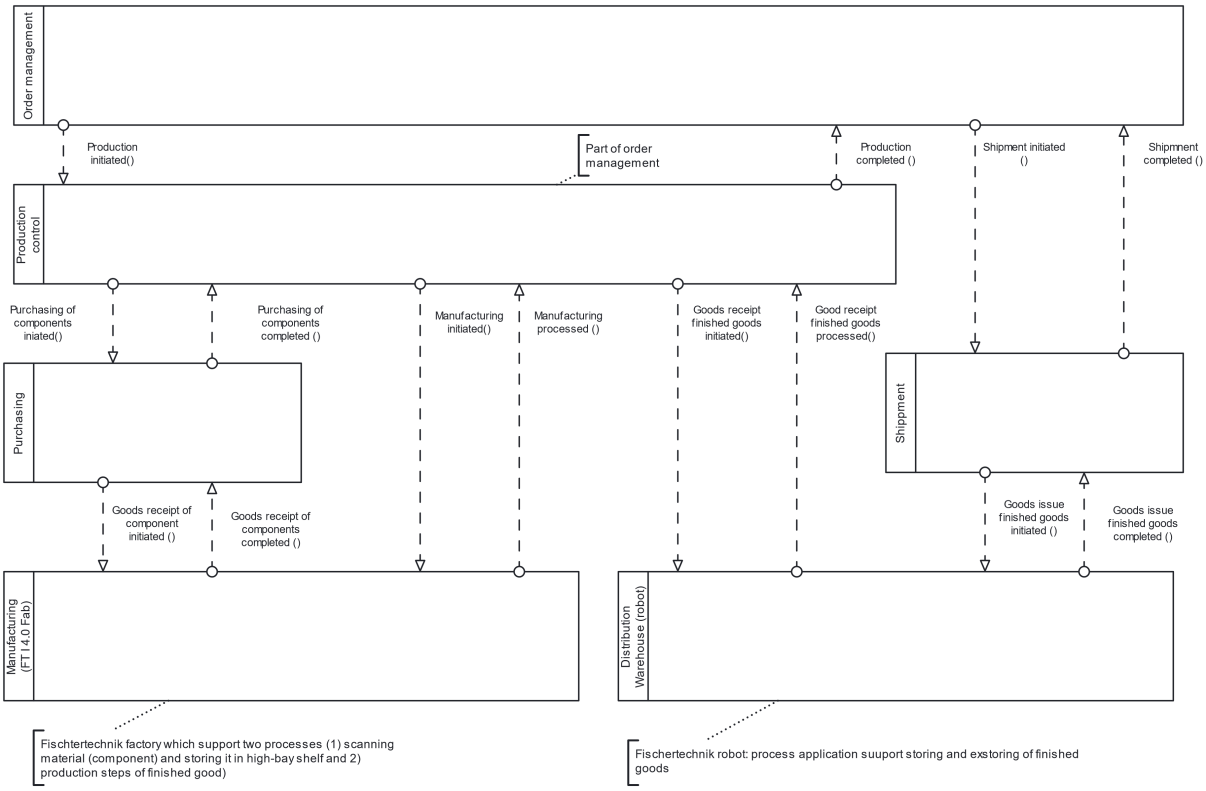

Figure 4.1.: Process landscape of the BPA Lab

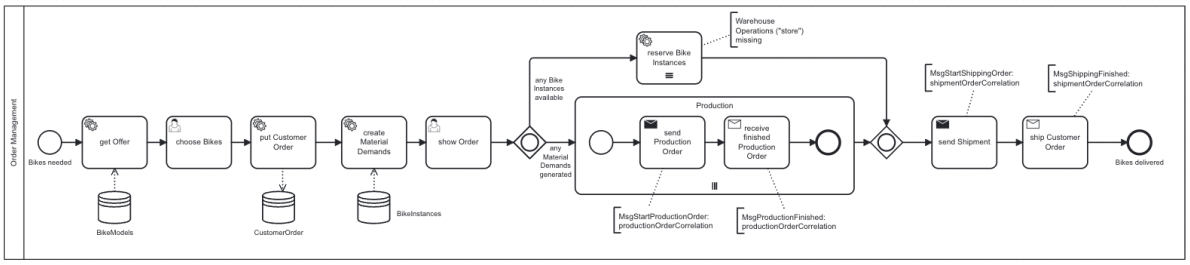

Figure 4.2.: Order management process

It starts when a customer order is entered into a web form, which is pre-filled with product data (bicycle models) from the database. During execution, the **availability of the ordered products** is checked.

Two main scenarios apply:

- **All products in stock:** the shipping process is triggered directly. Once shipping completes successfully, the process is finished.
- **One or more products out of stock:** for each unavailable product, the production control process is triggered, which creates a **production order**. The production order specifies the required components.
  - If the components are also unavailable, a **purchasing process** is started first.
  - Otherwise, the **manufacturing process** is triggered immediately.

Once the missing products are manufactured, shipping is initiated.

4.3. Production control

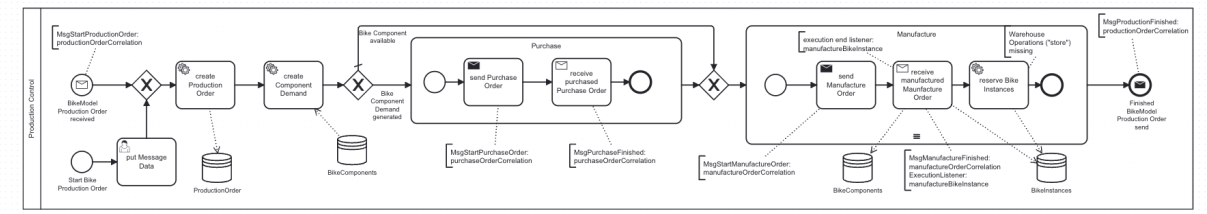

Figure 4.3.: Production control process

The production control process manages the creation and execution of **one production order for one product**. It determines the component demand and — if needed — triggers a purchasing process to procure the missing components. Once components are available, it triggers the actual manufacturing process.

4.4. Purchasing

The purchasing process manages the procurement of required components. It starts whenever a component is needed and creates a **purchase order**.

The purchase order can be adjusted by the user — for example by selecting a specific vendor. Materials are then stored and stock levels are updated. Upon user confirmation (or after a configurable timeout), the stock level is increased in the inventory database.

4.5. Manufacturing

The manufacturing process governs the **physical manufacturing simulation** on the Fischertechnik factory for **one item of one product type**.

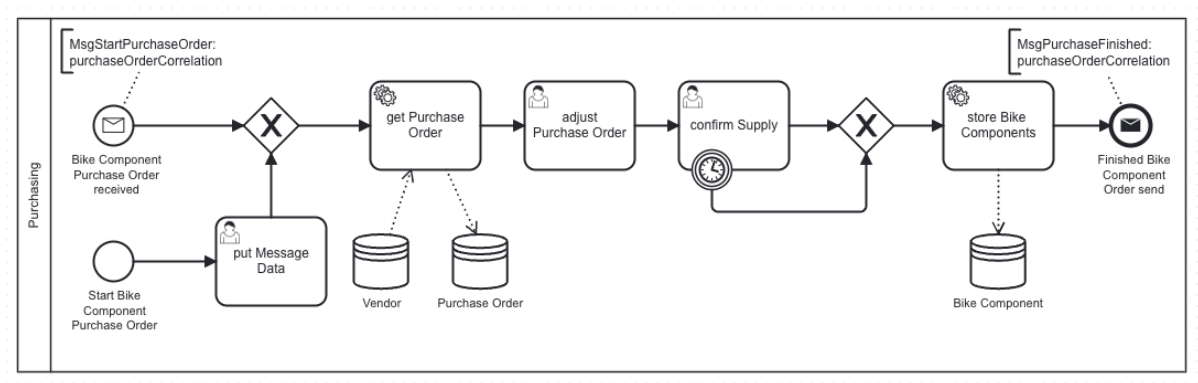

Figure 4.4.: Purchasing process

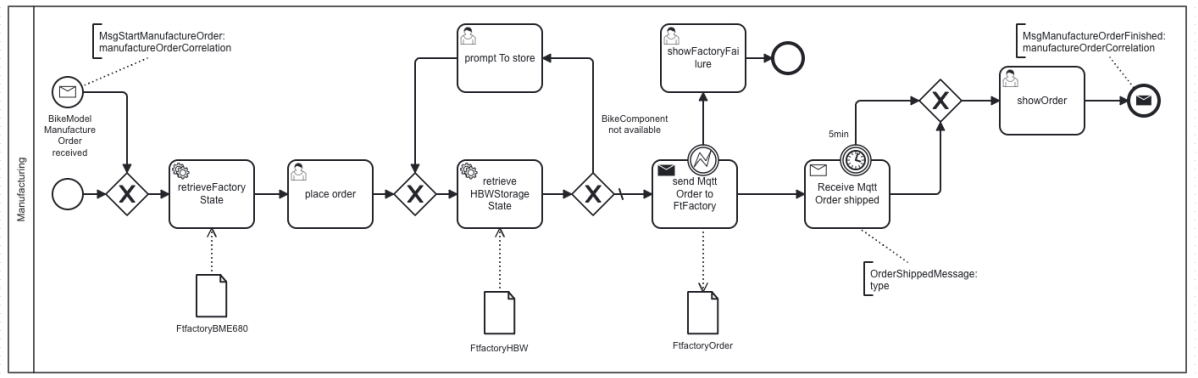

Figure 4.5.: Manufacturing process

It starts after a production order has been created by the production control process and sufficient components are available.

The process must place a manufacturing order that produces a new bicycle from the specified material. Before that, however, it must check whether a **physical workpiece** is already available in the warehouse of the Fischertechnik factory. If not, a physical workpiece must first be transferred to the factory so that the control program can store and recognise it. Once the bicycle has been produced from the material (workpiece), the process is complete.

4.6. Shipment

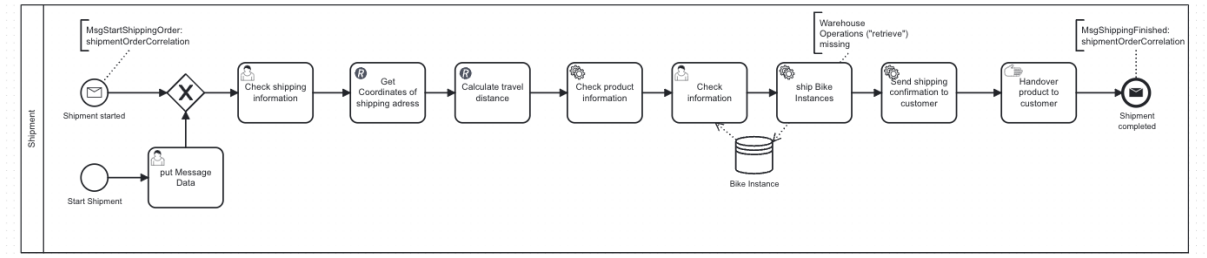

Figure 4.6.: Shipment process

The shipment process manages the dispatch of finished goods to the customer. It is **only** triggered if a product is in stock and ready to be shipped.

The flow is:

1. Shipping data is checked.
2. The distance and duration from the warehouse to the delivery address are computed — the **Openrouteservice API** is used for this.
3. The logistics sub-process retrieves the customer order from the warehouse.
4. A message is sent to the customer to inform them about the dispatch.
5. Upon handover of the order to the customer, both the shipping process and the surrounding end-to-end process instance are completed.

4.7. Warehouse operations

The warehouse operations process governs the flow of finished goods in a distribution warehouse — represented physically by the Fischertechnik warehouse robot.

It starts when a **transfer order** for the warehouse is received from another business process. A transfer order can be either an instruction to **store products** in the warehouse or to **retrieve products** from it.

4.8. Teaching note

Process landscape as a teaching tool

This process landscape works well to introduce key BPMN concepts in context:

- **call activities** (e.g. order management calling production control),
- **message-based communication** between long-running processes,
- **boundary events** to handle exceptions raised by job workers,
- **multi-instance activities** for handling several products in a single order.

By stepping through the running system together with students, abstract notation suddenly becomes concrete.

5. Data and Data Sources

> **Learning objectives**
>
> After reading this chapter, you will be able to:
>
> - identify the main data sources in the BPA Lab,
> - describe the role of the relational database in the overall architecture,
> - explain how process and event data become the foundation for process mining.

5.1. Overview of data sources

The BPA Lab integrates data from several sources — from **master data** in a relational database, via **process state** maintained by the workflow engine, to **sensor and telemetry data** from the Fischertechnik components.

The following figure gives an overview of the main data sources and the kinds of data they hold:

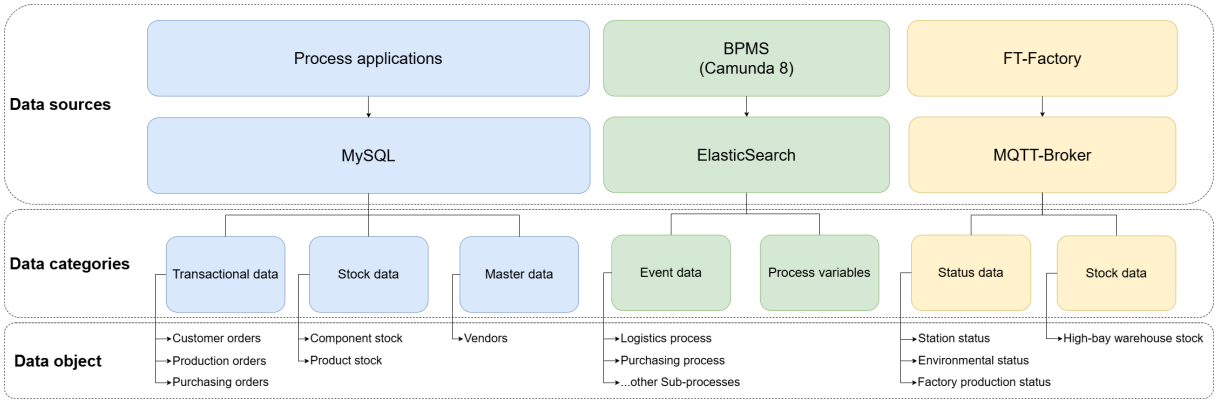

Figure 5.1.: Data overview in the BPA Lab

5.2. The MySQL data model

The following diagram shows the concrete data model of the MySQL database, including the entities and attributes of the current implementation:

The database serves as the central store for:

- **Product master data** — bicycle models and their components,
- **Customer data** and delivery addresses,
- **Stock levels** for components and finished goods,

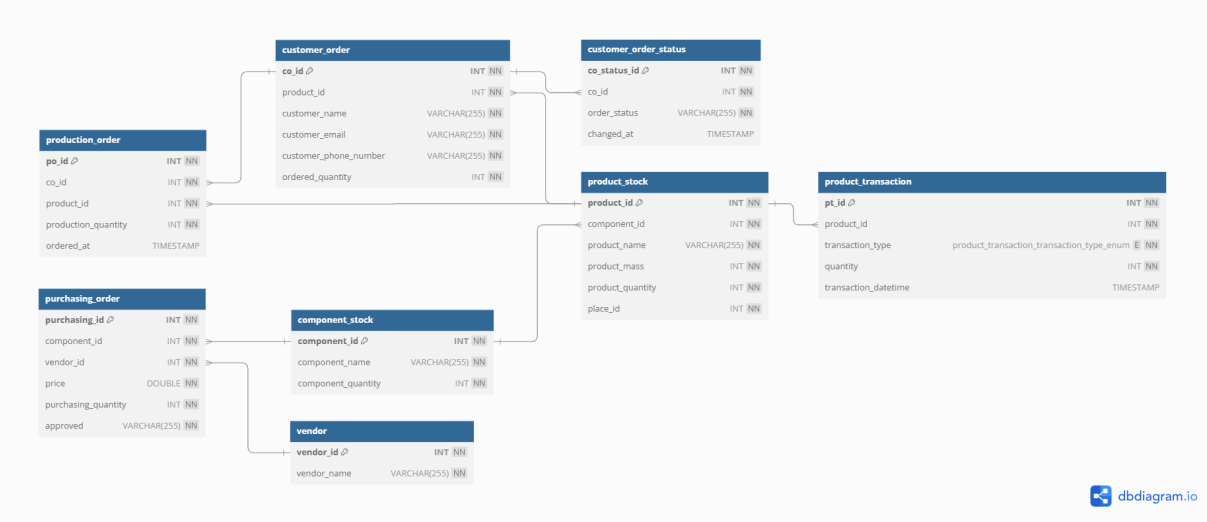

Figure 5.2.: MySQL data model

- **Orders** — customer orders, production orders, purchase orders — together with their status history,
- **Vendor data**.

5.3. Sample and master data

Documentation and sample data of the Fischertechnik factory are available in the `Sample_data` folder of the documentation repository. This data serves as:

- **initial master data** when setting up a new instance of the BPA Lab,
- **test data** for developing new components,
- a **demonstration baseline** for teaching sessions and presentations.

5.4. Process data and the foundation for process mining

In addition to persistent master data, the running processes generate substantial amounts of **process data**:

- **Camunda 8** stores process instance data, variables, tokens, and lifecycle events.
- **MQTT messages** record the communication between job workers and controllers.
- **Job worker logs** capture technical operations and exceptions.

This event data is the foundation for **process mining** — a central teaching and demonstration component of the BPA Lab. Process mining allows students to:

- **discover** the actual flow of processes from event logs (and compare them to the modelled flow),
- **check conformance** between modelled and actual behaviour,
- **analyse performance** — durations, bottlenecks, rework loops.

5.5. Data lifecycle and analytics

Looking at the data lifecycle as a whole:

1. **Master data** is loaded once when the lab is set up and updated when new products, customers, or vendors are added.
2. **Operational data** — orders, stock movements, production orders — is created continuously as processes run.
3. **Event data** is emitted by the workflow engine, the job workers, and the MQTT broker as side effects of process execution.
4. **Analytical data** is derived from the above by export and transformation pipelines, then fed into process-mining and BI tools.

This layered view of data is itself a teaching opportunity: it mirrors the classical separation of OLTP and OLAP systems that students will encounter in industry.

6. Software Architecture

Learning objectives

After reading this chapter, you will be able to:

- name the high-level requirements that shaped the BPA Lab architecture,
- describe the architecture using the C4 model,
- relate individual architectural choices to the requirements that motivated them,
- read and write lightweight Architecture Decision Records (ADRs),
- explain the rationale behind the central architectural decisions of the BPA Lab.

Status

The architecture is under continuous development. Several aspects are still being discussed. This chapter captures the current state.

6.1. Requirements

The following high-level requirements were gathered at the very beginning of the BPA Lab project (originally as part of a thesis project). They served as the basis for the first software architecture and remain the reference point against which architectural choices are evaluated.

6.1.1. R1 — Independent operation of components

The individual components and processes should be able to run independently of each other. Each process must be startable on its own, without depending on a triggering instance from a higher layer.

6.1.2. R2 — Integration of external IT systems

The system must allow integration with existing IT components — for example with the web application used for order capture, or with tools that flexibly extract data for process mining and analytics.

6.1.3. R3 — Clear and distinct responsibilities

The responsibilities of individual components must be clearly defined and demarcated from one another, so that the system remains understandable as it grows.

6.1.4. R4 — Easy extensibility

The system must be easy to extend with additional, self-developed components.

6.1.5. R5 — Interchangeability and independent evolution

Components must be developable, deployable, and replaceable independently of one another.

6.1.6. R6 — Comprehensive and flexible process data capture

Process data must be comprehensively capturable. It must be stored and made flexibly available for a range of analytical use cases — in particular process mining.

6.1.7. R7 — User-friendliness (operator perspective)

The user interface must be intuitive and easy to understand, ensuring a positive user experience for those interacting with the lab during demonstrations and student sessions.

6.1.8. R8 — Easy installation and low infrastructure requirements (developer perspective)

The system must be easy to install. Students must be able to host their own instances on standard laptops without specialised infrastructure.

6.1.9. R9 — Robustness

The architecture must be robust. Failures must be minimised and reliable functionality must be ensured under a wide range of operating conditions.

6.1.10. R10 — Maintainability and operation

The system must be easy to maintain over its lifetime.

6.2. Architecture in the C4 model

The architecture is documented using the **C4 model** (Brown 2018) — a notation for describing software systems at different levels of abstraction: Context, Container, Component, and Code. Note that *containers* in the C4 sense are **not** the same as Docker containers.

6.2.1. Context diagram

The context diagram shows the BPA Lab in relation to its users and external systems.

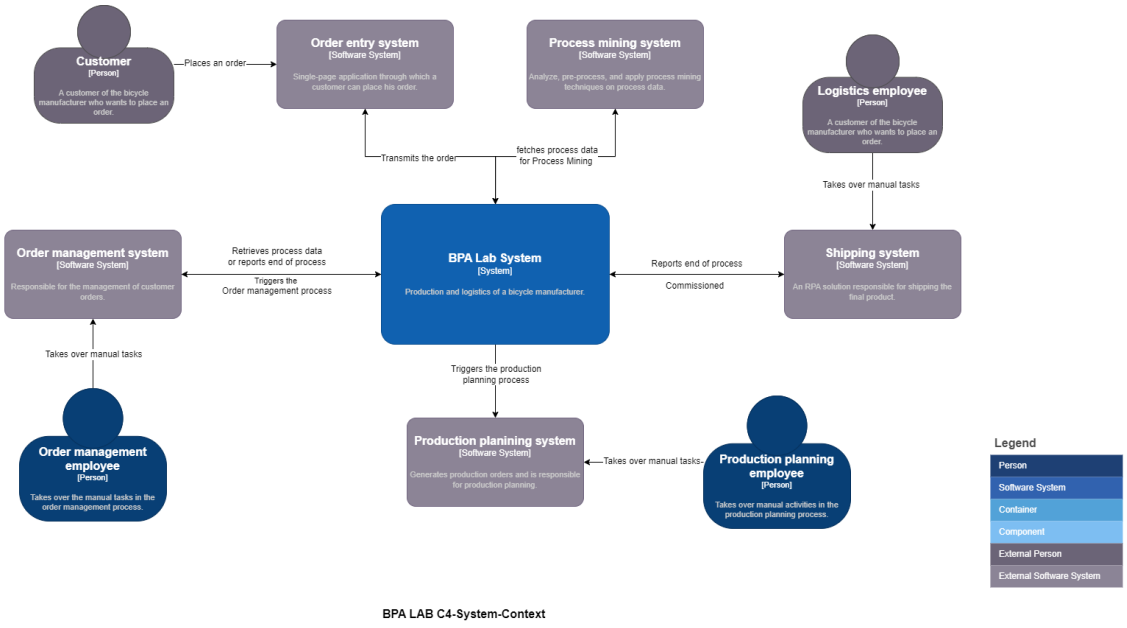

Figure 6.1.: C4 context diagram

6.2.2. Container diagram

The container diagram shows the main technical building blocks of the BPA Lab and how they interact.

6.3. Mapping requirements to architecture

The following table illustrates how individual requirements are addressed by architectural choices. It is not exhaustive — but it shows the line of reasoning from requirement to design.

Requirement	Realisation in the architecture
R1 — Independent operation	Loose coupling via MQTT and gRPC; self-contained job workers
R2 — Integration of external systems	Standard protocols (REST, MQTT, gRPC); Camunda connector concept
R3 — Clear responsibilities	Domain-driven separation of job workers and process models
R6 — Process data capture	Camunda 8 as central event hub; MySQL for operational data
R8 — Easy installation	Docker Compose setup; self-managed Camunda 8 Core
R5/R9 — Interchangeability / robustness	Asynchronous messaging; clear interfaces between layers

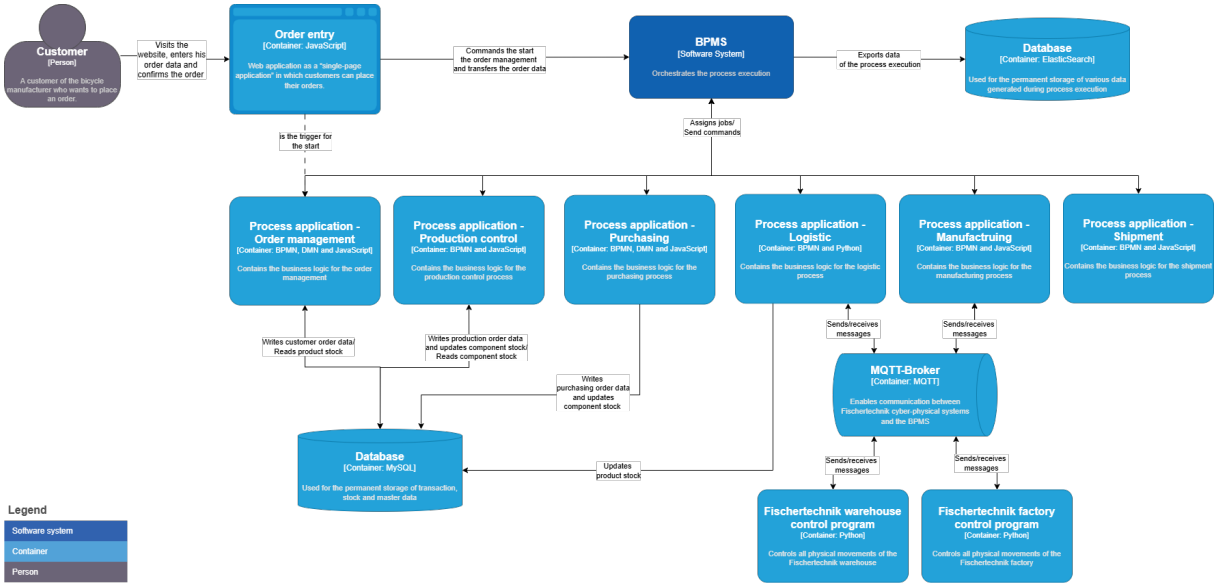

Figure 6.2.: C4 container diagram

6.4. Architecture decisions

The remainder of this chapter records selected architecture questions raised during the BPA Lab project, together with the corresponding decisions and justifications. The list is **not exhaustive** and is updated as the lab evolves.

Each decision follows a lightweight **ADR** (Architecture Decision Record) schema: context, decision, and rationale are captured succinctly.

6.4.1. ADR-001 — MQTT broker for job-worker–controller communication

Status: Decided.

Decision: A MQTT broker is used for communication between process applications (job workers) and controllers.

Rationale:

- decoupling of senders and receivers through the publish-subscribe pattern,
- MQTT is a well-established standard protocol in the IoT space,
- lightweight footprint, well suited to the hardware components in use.

6.4.2. ADR-002 — Message Send/Receive Tasks instead of Events for inter-process communication

Status: Decided.

Decision: Inter-process communication is modelled in BPMN using **Message Send Tasks** and **Message Receive Tasks**, not events.

Rationale:

- Tasks allow the use of **boundary events** — for example to handle errors raised by job workers,
- complex error paths and timeouts are easier to express.

6.4.3. ADR-003 — Messaging between processes and process applications

Status: Decided.

Decision: Communication between processes and process applications is implemented using messages as well.

Rationale:

- a simple, uniform solution,
- consistent with the communication pattern between process applications and process control.

6.4.4. ADR-004 — Camunda 8 self-managed with Docker Compose (Camunda 8 Core)

Status: Decided.

Decision: The self-managed variant of Camunda 8 is used in combination with Docker Compose and Camunda 8 Core.

Rationale:

- avoids interferences between contributors during the development phase,
- development and production-like environments are identical,
- Kubernetes is intentionally not used — the BPA Lab is not a production environment in the classical sense.

Open question: Performance with Docker and the resulting number of containers needs to be verified as the system grows.

6.4.5. ADR-005 — Job worker for sending emails

Status: Decided (reversible).

Decision: Email sending is implemented through a dedicated job worker rather than via the SendGrid connector.

Rationale:

- issues encountered with the SendGrid connector at the time of the decision,
- an existing solution was already in place in the project,
- to be re-evaluated as Camunda 8 connectors continue to evolve.

6.4.6. ADR-006 — Job worker for database transactions; MySQL in a Docker container

Status: Decided.

Decision: Database transactions are handled by a dedicated job worker; the MySQL database runs in a Docker container.

Rationale:

- existing solution available in the project,
- greater flexibility than the connectors available at the time of the decision.

6.5. Reflection questions

For your own consideration

1. ADR-001 selects MQTT. What **alternative protocols** could have been considered, and what trade-offs would they imply?
2. ADR-005 is explicitly marked as reversible. Which **criteria** would you use to decide whether to switch back to a connector-based solution?
3. ADR-004 deliberately avoids Kubernetes. Under which circumstances would that decision need to be revisited?

6.6. Going deeper

Further reading

The original sources on the C4 model — including examples, decision records, and tools like Structurizr — are available at https://c4model.com/. For a deeper dive into Domain-Driven Design as the principle behind the modular separation, see Evans (Evans 2003).

7. Projects

> **Purpose of this chapter**
>
> The BPA Lab is not only an artefact — it is also a vehicle for **research and teaching projects**. This chapter collects the projects conducted with, or around, the BPA Lab. Each entry summarises the project's motivation, approach, and outcomes, and links to the corresponding publication(s).

7.1. Teaching Business Process Automation in a Learning Factory (SoTL)

Authors: Matthias Zapp, Saphira Gonzalez & Domenic Gonzalez (TH Köln) **Publication:** *Forschung und Innovation in der Hochschulbildung*, Nr. 25, 2025, Research Paper **Publisher:** Cologne Open Science (COS), TH Köln **DOI:** 10.57684/COS-1326 **Type:** Scholarship of Teaching and Learning (SoTL) publication (Coleman et al. 2023)

7.1.1. Background and motivation

Business Process Automation (BPA) is a socio-technical challenge that requires a diverse set of competences — **technical** (process modelling in BPMN, implementing features in a BPMS, etc.), **methodological** (planning and running automation projects), as well as **social and personal** competences for collaborating with stakeholders from technical and business backgrounds (Nerdinger et al. 2008; Schaper 2012).

To teach this combination of skills, the bachelor's programme in Business Information Systems at TH Köln uses **Project-Based and Problem-Based Learning (PBL)** (Barrows 1996; Chow 2021; Santos et al. 2023). Students work in teams of around five to six people on automation projects using a BPMS — an approach that creates active learning and practical problem-solving, similar to PBL formats described in the literature (Bandara et al. 2010).

However, the authors observed a **persistent gap**: in regular bachelor's courses with around 100 students, project tasks must be simplified so they remain manageable. This unavoidable simplification means students often miss the experience of **integrating automation into a realistic enterprise IT landscape**. A telling classroom observation: on a final-presentation day, one student team was surprised by another team that demonstrated triggering a BPMS from a simulated company web portal — a feature mentioned in lectures, but absent from most project tasks because of the limits of a simplified setting.

7.1.2. Research question

The paper asks:

> *"To what extent can the use of a learning factory with software and hardware components improve project-oriented and problem-oriented teaching in the field of business process automation?"*

The authors draw on the well-established concept of **learning factories** from engineering education — small, realistic production environments that bridge theoretical knowledge and industrial practice (Abele et al. 2024). While most learning factories are physically large and centred on manufacturing, the BPA Lab is deliberately small-scale, with a stronger focus on **software systems and components** than on hardware.

7.1.3. The teaching concept

The proposed teaching concept extends the existing PBL format. Instead of working on isolated, simplified scenarios, student teams work on project tasks set in the **fictitious enterprise** represented by the BPA Lab — a customer-specific bicycle manufacturer with end-to-end processes for order processing, production, and shipping. The lab's process, decision, and data models together form a **"digital twin"** of this business scenario.

An illustrative example given in the paper: a project task to design and implement a **shipment process for special goods**, which requires integrating a software component of the BPA Lab dealing with shipment planning (using an external API for route planning) and integrating a **software-hardware module** — the Fischertechnik warehouse robot — to realise the connection with a warehouse management system.

Organisationally, milestone meetings with a "fictitious client" who asks critical questions and gives constructive feedback are added on top of the existing PBL structure.

7.1.4. Preliminary evaluation

The evaluation was performed in **two phases**.

Phase 1 — Interviews with bachelor's students. Students from two previous course cycles were presented with the proposed teaching concept and asked to imagine integrating the learning factory into their project. They reacted very positively, using terms like *"very important", "inspiring", "exciting"* and *"very interesting"*. They emphasised that the lab could make BPA *"less abstract and more tangible"*, deepen understanding through hands-on experience, and promote transfer thinking and motivation. They also identified several requirements: clear instructions and materials, sufficient teaching support, the risk of technical difficulties with hardware, and the importance of integrating the lab thoughtfully into existing modules to avoid overload.

Phase 2 — Pilot project with master's students. A project simulation was carried out with **eight master's students** from the *Digital Sciences* programme. Compared to bachelor's students, the participants had comparable BPM knowledge but stronger technical skills and software-development experience. They were asked to solve a project task and assess — from the perspective of bachelor's students — whether the concept could realistically be implemented at that level.

Students overall perceived the opportunities of the concept as **positive**: the practical experience of working with real BPM tools, the learning-by-doing approach, and teamwork on individual challenges were all valued. At the same time, the evaluation revealed **substantial challenges**: complexity of installing and configuring the lab's components, individual problems with hardware/software/network setup, and difficulties in dealing with certain BPMS concepts. Students requested both **lab-specific documentation** and **basic background material** on the underlying technologies.

7.1.5. Conclusions and consequences for teaching

The authors arrive at a **nuanced conclusion**: the teaching concept as evaluated is **not directly applicable** to the bachelor's course at TH Köln with ~100 participants. The complexity of using the full BPA Lab — combining software development, network and hardware setup — exceeds what can reasonably be supported in a large bachelor's course.

Two practical consequences follow:

1. **For the existing bachelor's course:** the BPA Lab can serve as a **realistic project background** that is demonstrated at the start of the project. The technical project task itself, however, should only be extended carefully — in particular, **integration of hardware components requires more prior knowledge and support** than is feasible. **Pure software components** of the BPA Lab are a more realistic extension.

2. **For future courses:** the lab has potential as the basis of **new elective course formats**, which by design can cover a wider range of learning objectives — from BPM and BPA through to IoT and Cyber-Physical Systems — and allow for a higher student workload.

For BPM educators in general, the paper provides early evidence on the *opportunities*, *challenges*, and *requirements* of using a small-scale learning factory in a BPA course — an approach that is interesting but demanding, both for students and for teaching staff.

7.1.6. Funding

The underlying teaching project *"Enhancing teaching in the field of business process automation through a learning factory"* was supported by the **Foundation for Innovation in Higher Education** as part of the **REDiEE** project at TH Köln.

7.1.7. Further reading

- Full paper: 10.57684/COS-1326 (open access, CC BY-NC-ND)
- TH Köln SoTL programme: https://www.th-koeln.de/hochschule/scholarship-of-teaching-and-learning-sotl_115776.php
- Series *Forschung und Innovation in der Hochschulbildung* (FIHB), Cologne Open Science

7.2. Human-Centred Design of Form-Based User Interfaces in Camunda 8

Author: Julian Will · **Supervisor:** Prof. Dr. Matthias Zapp · **Second supervisor:** Prof. Dr. Raphaela Groten **Type:** Bachelor's thesis, Business Information Systems, TH Köln · **Submitted:** January 2026

7.2.1. Contribution to the BPA Lab

This thesis delivers a **redesigned set of user-task forms** for the BPA Lab demonstration factory, replacing the previously rudimentary Camunda 8 forms with usability-tested versions. Concretely, the work contributes:

- A **consistent form layout template** — TH Köln header with form title and job role, plus colour-coded message blocks (blue for guidance, red/orange/green for error/warning/confirmation) implemented as reusable HTML views.
- **BPM-specific usability guidelines** distilled from Nielsen's heuristics, Norman's design principles, and ISO 9241-11/210 — directly applicable to future student projects using Camunda forms.
- **Redesigned or newly created forms** covering the full bicycle-manufacturer scenario, including four entirely new forms (*Confirm Supply*, *Quality Control*, *Missing Workpiece*, *Factory Failure*) that close previously empty user-task slots.
- Further **technical extension** e.g. in the shipment process.
- An **empirical baseline** from a usability test with five participants (8 forms, 40 runs, mixed-methods: think-aloud, completion time, success rate, Likert questionnaire) that future revisions can be measured against.

7.3. Further projects

Additional research and teaching projects conducted with the BPA Lab will be listed here as they are completed and published. Typical types of entries include:

- **Bachelor's and master's theses** that use the BPA Lab as their experimental platform,
- **Student projects** that extend the lab with new components or integrate new technologies,
- **Research publications** that use the lab to validate concepts in business process automation, process mining, or related fields,
- **Industry collaborations** that build on or contribute to the lab.

> **Contributing a project entry**
>
> If you have conducted a project using the BPA Lab and would like it listed here, please open an issue or pull request in the book repository. A short summary in the same structure as the SoTL entry above — *motivation*, *approach*, *outcomes*, *publication link* — is sufficient.

A. Components and Sister Repositories

This appendix gives an overview of the repositories that together make up the BPA Lab ecosystem, and lists the main hardware and software components.

A.1. Main repositories

A.1.1. bpa_lab_docs

https://github.com/BpaLabTHCologne/bpa_lab_docs

Role: central documentation and architecture of the BPA Lab as a demonstration factory. The source of the content in this book.

Contains:

- overarching architecture diagrams,
- sample data of the Fischertechnik factory,
- BPMN process models (overview),
- C4 architecture diagrams,
- updated graphics.

A.1.2. bpa_lab_demonstration_factory

https://github.com/BpaLabTHCologne/bpa_lab_demonstration_factory

Role: current implementation of the demonstration factory. A complete, runnable setup based on Docker.

Contains:

- self-managed Camunda 8 setup,
- all job worker implementations (Java),
- BPMN and DMN models of the business processes,
- controller implementations (Python),
- `docker-compose.yml` for local deployment.

A.1.3. bpa_lab_student_docs

https://github.com/BpaLabTHCologne/bpa_lab_student_docs/wiki

Role: documentation for student projects. Describes how individual components (e.g. the warehouse robot) can be used in self-developed process implementations.

Contains:

- per-component how-to guides,
- example projects,
- notes on setting up subsystems independently.

A.2. Hardware components

Component	Purpose	Control
Fischertechnik Industry 4.0 factory	Goods receipt and manufacturing	Python controller, MQTT
Fischertechnik warehouse robot	Storage and retrieval in the distribution warehouse	Python controller, MQTT
IoT devices	Sensor data and supplementary telemetry	Python controller, MQTT

A.3. Software components

Component	Technology	Main responsibility
Camunda 8 (self-managed)	BPMS, based on Zeebe	Workflow engine and process orchestration
Job workers	Java, gRPC	Execution of service tasks
Controllers	Python	Hardware control
MQTT broker	e.g. Mosquitto	Asynchronous communication
MySQL	Relational database	Master data and stock data
Web application	Frontend	Order capture and user interaction
Openrouteservice	External API	Distance and routing calculation in the shipment process

A.4. Further reading

As the BPA Lab is under continuous development, it is always worth checking the respective repository READMEs for the most current information.

B. Glossary

This appendix explains the technical terms used throughout the book.

ADR (Architecture Decision Record) Lightweight documentation format for recording architectural decisions in a structured way — context, decision, and rationale.

BPA (Business Process Automation) The use of technology to automate the execution of business processes.

BPMN (Business Process Model and Notation) A standard of the Object Management Group (OMG) for graphical modelling of business processes. Current version: 2.0.2 (Object Management Group 2014).

BPMS (Business Process Management System) A platform for modelling, executing, monitoring, and optimising business processes. The BPA Lab uses Camunda 8.

Camunda 8 A cloud-native BPMS built around the Zeebe workflow engine. The BPA Lab uses the *self-managed* variant (Camunda Services GmbH 2026).

Controller In the BPA Lab, a software component that directly controls hardware (Fischertechnik components, IoT devices). Predominantly implemented in Python.

C4 model A notation for visualising software architecture at four levels of abstraction: Context, Container, Component, Code. Introduced by Simon Brown — https://c4model.com/ (Brown 2018).

DDD (Domain-Driven Design) An approach to software development that places the business domain at the centre. The guiding principle for the modular decomposition of the BPA Lab (Evans 2003).

DMN (Decision Model and Notation) An OMG standard for modelling business decisions (Object Management Group 2024). Supported in Camunda 8 as a complement to BPMN.

End-to-end process A business process viewed from its triggering event through to its final business outcome — in the BPA Lab, from the arrival of a customer order to the delivery of a bicycle.

Fischertechnik Industry 4.0 factory A modular model of an Industry-4.0 factory by Fischertechnik. In the BPA Lab, it serves as the physical demonstration system.

gRPC A high-performance Remote Procedure Call framework originated at Google. In the BPA Lab, Camunda 8 uses gRPC to communicate with job workers.

Happy path The path through a process in which no errors, exceptions, or edge cases occur.

IoT (Internet of Things) The networking of physical devices with the internet and with each other.

Job worker In Camunda 8, an external application that picks up service tasks from the workflow and executes them. In the BPA Lab, job workers are implemented in Java.

MQTT (Message Queuing Telemetry Transport) A lightweight publish-subscribe protocol widely used in the IoT space. In the BPA Lab, MQTT is the communication protocol between job workers and controllers.

Openrouteservice A public API for routing and distance calculation based on OpenStreetMap data. Used in the BPA Lab's shipment process.

Process mining Data-driven analysis of business processes based on event logs from information systems. A central teaching and demonstration component in the BPA Lab (Aalst 2016).

Process application In the BPA Lab, a logical grouping of job workers and process models belonging to one business domain (e.g. *Order Management*).

Production order An order to manufacture a specific product. In the BPA Lab, one production order is created per product line in a customer order.

RPA (Robotic Process Automation) A technology to automate manual, repetitive tasks in existing applications via their user interface.

Workflow engine A software component that executes process models (e.g. expressed in BPMN), manages the state of process instances, and distributes work to humans or job workers.

XES (eXtensible Event Stream) An IEEE standard for event logs in process mining — a format for the exchange of process event data between systems.

C. References

Aalst, Wil M. P. van der. 2016. *Process Mining: Data Science in Action*. 2nd ed. Springer. https://doi.org/10.1007/978-3-662-49851-4.

Abele, Eberhard, Joachim Metternich, Michael Tisch, and Antonio Kreß. 2024. *Learning Factories: Featuring New Concepts, Guidelines, Worldwide Best-Practice Examples*. 2nd ed. Springer. https://doi.org/10.1007/978-3-031-46428-7.

Bandara, W., D. R. Chand, A. M. Chircu, et al. 2010. "Business Process Management Education in Academia: Status, Challenges, and Recommendations." *Communications of the Association for Information Systems* 27: 748–50. https://doi.org/10.17705/1CAIS.02741.

Barrows, Howard S. 1996. "Problem-Based Learning in Medicine and Beyond: A Brief Overview." *New Directions for Teaching and Learning* 1996 (68): 3–12. https://doi.org/10.1002/tl.37219966804.

Brown, Simon. 2018. *The C4 Model for Visualising Software Architecture*. Leanpub. https://c4model.com/.

Camunda Services GmbH. 2026. *Camunda 8 Documentation*. https://docs.camunda.io/.

Chow, W. 2021. "Teaching Business Process Management with a Flipped-Classroom and Problem-Based Learning Approach." *2021 International Conference on Engineering, Technology & Education (TALE)*. https://doi.org/10.1109/TALE52509.2021.9678885.

Coleman, K., D. Uzhegova, B. Blaher, and S. Arkoudis, eds. 2023. *The Educational Turn: Rethinking the Scholarship of Teaching and Learning in Higher Education*. Springer. https://doi.org/10.1007/978-981-19-8951-3.

Evans, Eric. 2003. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Nerdinger, F., G. Blickle, and N. Schaper. 2008. *Arbeits- Und Organisationspsychologie*. Springer Medizin. https://doi.org/10.1007/978-3-540-74705-5.

Object Management Group. 2014. *Business Process Model and Notation (BPMN), Version 2.0.2*. https://www.omg.org/spec/BPMN/.

Object Management Group. 2024. *Decision Model and Notation (DMN)*. https://www.omg.org/spec/DMN/.

Santos, S. C. dos, J. Vilela, and A. Vasconcelos. 2023. "Promoting Professional Competencies Through Interdisciplinary PBL: An Experience Report in Computing Higher Education." *2023*

Frontiers in Education Conference (FIE). https://doi.org/10.1109/FIE58773.2023.10343050.

Schaper, N. 2012. *Fachgutachten Zur Kompetenzorientierung in Studium Und Lehre*. Hochschulrektorenkonferenz, Projekt Nexus. https://www.hrk-nexus.de/fileadmin/redaktion/hrk-nexus/07-Downloads/07-02-Publikationen/fachgutachten_kompetenzorientierung.pdf.